

EXERCISE HANDOUT

Writing Clean, Testable, Modular Code

Refactoring Exercise: The Order Processor · Fidelity, Gurgaon Fresher Training · Language: Python

Learning Objectives

By completing this exercise, you will practice:

- Renaming variables and functions to reveal intent
- Breaking a large function into small, single-purpose functions
- Separating business logic from I/O (side effects)
- Using dependency injection to make code swappable and testable
- Writing unit tests that don't depend on a real database or network call

The Scenario

Below is a real (if exaggerated) module from a small e-commerce system: `order_processor.py`. It calculates an order total, applies a loyalty discount, saves the order, and emails the customer. It works — but it breaks nearly every principle covered in today's session. Your job is to refactor it into clean, testable, modular code without changing its observable behavior.

Starter Code

Save this as `order_processor.py` and use it as your starting point.

```
# order_processor.py
# TODO: this module works, but it needs a serious cleanup before it ships.

db = []

def process(o):
    if o['items'] == None or len(o['items']) == 0:
        print('error: no items')
        return None
    t = 0
    for i in o['items']:
        t = t + i['price'] * i['qty']
    d = 0
    if o['cust']['orders'] > 10:
        d = 0.15
    else:
        if o['cust']['orders'] > 5:
            d = 0.1
        else:
            if o['cust']['orders'] > 0:
                d = 0.05
            else:
                d = 0
```

```

    t2 = t - (t * d)
    if t2 > 500:
        t2 = t2 - 20
    db.append({'id': o['id'], 'total': t2})
    print('Order ' + str(o['id']) + ' saved. Total: ' + str(t2))
    if o['cust']['email'] != None:
        print('Emailing ' + o['cust']['email'] + ': your order total is ' +
str(t2))
    return t2

def process(o): # NOTE: yes, there are two functions named process() in this
file
    return process(o)

```

Your Tasks

1. Rename. Give every variable, function, and parameter a name that explains what it holds or does, without needing a comment.
2. Split. Break the single process() function into small functions, each with one job — for example: validate_order(), calculate_subtotal(), calculate_discount_rate(), apply_bulk_discount(), and a coordinating function that calls them in order.
3. Modularize. Separate the pure calculation logic (subtotal, discount, final total — no I/O) from the side-effecting steps (saving the order, notifying the customer). Consider organizing these into separate classes or modules, e.g. pricing.py and notifications.py.
4. Inject dependencies. Introduce an OrderProcessor class (or equivalent) whose constructor accepts the data store and the notifier as parameters, instead of using a hard-coded global list and print(). Provide simple interfaces/classes for these so they can be swapped for fakes in tests.
5. Remove the bug. Fix the duplicate process() definition and the infinite recursion it causes.
6. Test. Write at least 5 unit tests using pytest that exercise the core logic (discount tiers, subtotal calculation, the bulk-order discount, and the empty-order validation error) using fake/in-memory implementations of the data store and notifier — no real database or console output required to run the tests.

Constraints

- Keep the same discount tiers and behavior: 5% after 1 order, 10% after 5 orders, 15% after 10 orders, and an additional ₹20 off any order total over ₹500 after the discount is applied.
- Do not use any external libraries beyond the Python standard library and pytest.
- Each function should be short enough to read without scrolling (roughly ≤ 15 lines is a good target).
- No function should have more than 3 parameters — group related values into a small class or dict if needed.

Deliverables

- order_processor.py — your refactored module
- test_order_processor.py — your unit tests
- A 3–5 line note at the top of order_processor.py explaining the key structural changes you made and why